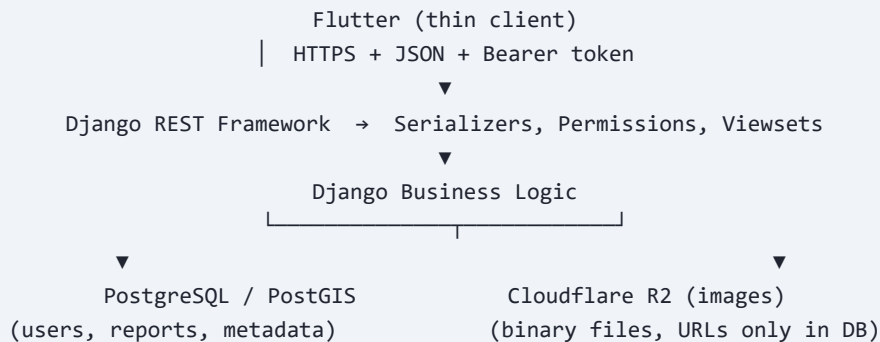


Fix My Grid — Master Tech Guide

Hackathon · Backend + System Architecture · Citizens report issues (potholes, broken lights, utilities) to the authority · v1.1 · Study top-to-bottom once, then drill the ⚡ **Say This** boxes out loud.

Product in one line: Fix My Grid lets a citizen snap/describe a civic issue, tag its GPS location, and submit it — the backend validates, stores, AI-grades, dedupes, and routes it to the right authority department.

1 · The Architecture (one picture)



⚡ **Say This:** "Flutter is the thin client — it only renders UI and makes HTTP calls. All logic lives server-side in Django. DRF is the API layer that translates JSON to Python. PostgreSQL stores structured data; R2 stores the binary images. That separation lets us scale each piece independently."

Why Django?

- **Batteries included** — auth, ORM, admin, migrations ship built-in; we don't assemble foundations in 24 hours.
- **DRF** — the #1 Python REST framework: serializers, permissions, throttling.
- **Security defaults** — protection against SQL injection, XSS, CSRF, clickjacking on by default.
- **Ecosystem** — django-storages, boto3, PostGIS all battle-tested.

Why PostgreSQL instead of MongoDB?

- **Relationships:** users *own* reports — a foreign key. Postgres enforces referential integrity; Mongo lets you store anything anywhere.
- **PostGIS:** geo queries ("potholes near me") need it.
- **ACID transactions:** report + image + severity must all succeed or all roll back — no half-written reports.
- **SQL is mature:** predictable, works with every analytics tool.

⚡ **Counter:** "MongoDB wins when the schema is unpredictable. Ours isn't — a report is a report. So a strongly-typed relational DB is the right tool."

Why PostGIS?

- We are a **location-based app** — spatial data is the core product.
- `ST_DWithin` / `ST_Distance` / `ST_Contains` : distance queries, bounding boxes, "within 2 km", "inside which district — all in one indexed SQL call.
- **GiST spatial indexes** keep searches fast at millions of rows.

- One database does everything — no separate geo engine bolted on.

2 · Authentication (how it works, step by step)

Authentication = prove who you are. Authorization = prove what you're allowed to do.

1. Flutter sends `POST /api/auth/login/` with username + password over **HTTPS** (never plain HTTP).
2. Django hashes the incoming password and compares to the stored hash → match? Issue a token.
3. Return `{"token": "abc123"}`. The app stores it securely and sends it on every request.
4. Every request carries `Authorization: Bearer <token>`.
5. DRF's auth backend validates the token, checks expiry, resolves `request.user`.
6. Permission classes then decide *who may call this endpoint*; object checks decide *who may touch this row*.

Token vs Session

	Session auth	Token auth (our choice)
State	Server stores session, sends cookie	Stateless — no server memory
Mobile fit	Cookies are awkward in apps	Header-based, app-friendly
Scaling	Sticky sessions hurt horizontal scale	Any server can serve any request
Revocation	Kill the session	Delete/expire the token

 **Say This:** "Token auth is stateless: login returns a token, the app sends it in the Authorization header, DRF resolves it per request. No session memory means we scale to many servers trivially."

Password hashing

- We never store passwords — we store a **salted hash**. Hashing is one-way; encryption is reversible. We don't encrypt passwords.
- **Salt** = random data per password → identical passwords produce different hashes; rainbow tables are useless.
- **Slow algorithms** (PBKDF2 / Argon2 / bcrypt) make brute force expensive per guess. Django defaults to PBKDF2 with per-user salt.
- Verify = hash the input and compare digests; the plaintext never touches the DB or logs.

Recite: "If the DB leaks, attackers get salted hashes, not passwords. We'd rotate tokens and force resets — and we never log or return hashes."

Permissions (layer 1 — endpoint level)

- `AllowAny` — login / register (public).
- `IsAuthenticated` — report endpoints. A logged-out call: 401.
- Custom classes — e.g. `IsAuthorityUser` for department dashboards.
- Enforced in the view layer, server-side — never in the Flutter UI, which anyone can bypass.

Object-level authorization (layer 2 — per row)

- **Queryset filtering:** `Report.objects.filter(owner=self.request.user)` — listing only ever returns your rows.
- **Object check:** `if report.owner != request.user: raise PermissionDenied` on detail/update/delete.

⚡ **The money line:** "The filter lives in the queryset, not the UI. Someone with curl can't read others' reports, because the query literally only selects rows where owner = the token's user."

User ownership

- The `owner` FK is set **server-side**: `serializer.save(owner=request.user)` .
- Client-supplied `owner` fields are declared `read_only` and ignored — it's structurally impossible to spoof.
- Trusting client `owner` would be a classic IDOR / mass-assignment vulnerability.

Input validation

- Flutter validates for **UX**; Django (serializers) validates for **security** — only the server's check is trustworthy.
- What it catches: wrong types, missing fields, max lengths, coordinate ranges (−90..90 / −180..180), file type, 5 MB cap, oversized payloads → 400 before anything touches a model.

CORS / CSRF

- **CORS**: a *browser* rule — which origins may call your API with JS. Native Flutter ignores CORS entirely; the real defense is auth + tokens.
- **CSRF**: forged requests riding a victim's *cookie*. We auth by header token, not cookie → the classic CSRF vector doesn't apply.
- We still set CORS to known origins and protect cookie-based login forms.

Environment variables / secrets

- Secrets (`SECRET_KEY` , DB password, R2 keys) live in env vars — never in code or git.
- Local: `.env` (gitignored). Production: Render's Environment tab injects them.
- `DEBUG=False` in prod — a debug-stack-trace page publicly leaked is a real breach.
- Rotating a secret = changing an env var + redeploy, no code change.

3 · Image Upload Lifecycle (the full journey)

1. Flutter builds `multipart/form-data` : image file + GPS lat/lng + description, over HTTPS.
2. DRF's `MultiPartParser` takes it apart; the serializer validates everything (size cap, types, ranges). Faster than base64, no JSON-binary hack.
3. Django writes the image to **R2** via **django-storages** (which calls **boto3**) at `reports/{id}/photo.jpg` .
4. Only the **URL/key** is stored in PostgreSQL — never the blob (databases are bad at blobs).
5. Optionally **presigned URLs**: server signs a short-lived URL, Flutter uploads directly to R2 → zero server bandwidth for the bytes; download links can expire for privacy.
6. Background worker (Celery) then runs AI severity + duplicate detection off the request path.

Multipart vs JSON

JSON is UTF-8 text — it can't carry binary. base64-encoding a 5 MB image → ~6.7 MB of text (33% overhead). Multipart streams raw bytes alongside the text fields. Django/DRF handle the parsing automatically.

R2 vs S3 vs Firebase

- **R2 is S3-compatible** (same API contract) but charges **zero egress fees** and rides Cloudflare's CDN — ideal for an image-heavy app.
- S3-compat means the same storage code works anywhere — switching providers is config, not a rewrite.
- **django-storages** = framework integration (your model field just works). **boto3** = the low-level client (used for presigning and custom object ops).
- **Firebase Storage** is fast to start but locks you into Google's stack and pricing.

Presigned URLs — how to explain

⚡ **Say This:** "Presigned URLs are short-lived, one-action, one-object signed URLs. For upload: Flutter PUTs straight to R2 so images never pass through our server. For download: links expire in minutes, so private images stay private. The egress is free — it never touches Django."

```
s3 = boto3.client("s3",
    endpoint_url=os.environ["R2_ENDPOINT"],
    aws_access_key_id=os.environ["R2_ACCESS_KEY"],
    aws_secret_access_key=os.environ["R2_SECRET_KEY"])
url = s3.generate_presigned_url(
    "put_object",
    Params={"Bucket": "pothole-images", "Key": f"reports/{id}/photo.jpg"},
    ExpiresIn=300)
```

4 · App ↔ Server Communication & Failure Behavior

How the app communicates

- Over **HTTPS** with **JSON** bodies. The app never touches the DB — the DB isn't on the public internet.
- Flutter repository pattern: widgets → repository → HTTP client → JSON → Dart models. UI never knows about networking.
- Errors are HTTP codes with structured JSON bodies; the app maps them to UI states (400 bad input, 401 re-login, 403 forbidden, 404 missing, 429 throttled, 503 down).

What happens if the server goes down

- **Client side:** retry with backoff, an offline queue for the in-progress report, clear user messaging ("saved — uploads when back online").
- **Server side:** Render health checks auto-restart a crashed service.
- **DB down:** API returns 503; Postgres ACID means no torn writes on recovery.
- **R2 down:** keep metadata, retry the image upload with idempotent unique keys — image and metadata writes are coordinated with retry, not one fragile transaction.

Scaling: 1,000 → 1,000,000 users (talk in layers)

Scale	What you add
1,000	One Django server + one Postgres + one R2 bucket.
10,000	CDN/cache in front of images · PgBouncer connection pooling · indexes on hot columns + GiST on location.
100,000	Horizontal Django scale behind a load balancer (stateless token auth = no sticky sessions) · Redis for queues/cache.
1,000,000	Read replicas (reads to replicas, writes to primary) · Celery workers take AI/dedup off the request path · partition by region · PostGIS keeps geo queries indexed.

⚡ **Say This:** "Django is stateless with token auth, so scaling is adding instances behind a load balancer. Bottleneck checkpoints: database (→ pooling + replicas), heavy jobs (→ worker queue), image egress (→ free on R2, CDN-cached)."

5 · Tech-by-Tech Reference (your full checklist)

Core

Python

The language of the backend. Dynamically typed, huge ecosystem. CPU-heavy work (AI scoring) is offloaded to workers because of the GIL — API threads stay I/O-bound and responsive.

⚡ **Say This:** "Python powers the whole backend — Django on top — chosen for ecosystem and build speed. The bottleneck isn't Python, it's queries and I/O; hot paths use PostGIS SQL, and AI work runs on background workers."

Django

Full web framework, MVT: models (tables), views (logic), URLs (routing), migrations (versioned schema), settings, middleware, admin. Middleware wraps every request (auth, CORS, logging).

⚡ **Say This:** "Django is the backbone — routing, auth, ORM, migrations and security defaults for free, so we spend time on business logic, not plumbing."

Django REST Framework

The 5 components: **Serializers** (JSON ↔ models + validation), **Views/Viewsets** (list/create/retrieve/update/destroy), **Routers** (auto URLs), **Permission classes**, **Auth backends**. Request flow: header → auth backend → permission class → serializer validate → view logic → serialized JSON + status code.

⚡ **Say This:** "DRF gives serializers for validation, permission classes for authorization, and viewsets that auto-generate our REST endpoints."

REST API

Method	URL	Action
POST	/api/auth/login/	login → token
GET	/api/reports/	list my reports
POST	/api/reports/	create (multipart)
GET	/api/reports/{id}/	one report
GET	/api/reports/nearby/?lat=&lng=&radius=	geo query

Stateless, resource-URLs, standard verbs, meaningful status codes.

HTTP / HTTPS

- Request anatomy: request line, headers (`Content-Type` , `Authorization`), body.
- HTTPS = HTTP + TLS: handshake, certificate from a CA, encrypted data. Hashes protect at-rest; HTTPS protects the password/token **in transit** — a WiFi sniffer reads plain HTTP instantly.
- We force SSL redirect + HSTS in production.

JSON

The API contract — structured, debuggable, natively parsed by Flutter. JSON can't carry binary → files go multipart, and only URLs live in JSON.

PostgreSQL

- Tables + typed columns, **foreign keys enforced** (no orphan rows), **ACID** (all-or-nothing writes), **indexes** (speed reads, trade write speed).
- ORM generates safe parameterized SQL: `SELECT * FROM reports WHERE owner_id = ?` — input is a parameter, never concatenated (that's the SQL-injection defense).
- Postgres vs MySQL: more features (JSONB, PostGIS) and stricter correctness — fits geo data.

Django ORM

- Python objects ↔ DB tables; lazy QuerySets execute SQL only when evaluated.
- Avoid **N+1**: `select_related('owner')` → one JOIN, one query.
- Raw SQL available (PostGIS-heavy report queries) when the ORM gets awkward — 95% stays in the ORM.

PostGIS

- Extension → spatial types (`Point`), SRID 4326 = WGS84 lat/long.
- `ST_DWithin(geom,p,m)` — nearby/duplicates · `ST_Distance` — meters · `ST_Contains` — which district (routing).
- GiST spatial indexes keep geo fast at scale; no per-row Python math.

Authentication & Security

- **AuthN vs AuthZ:** "who are you?" (token) vs "may you do this?" (permission). An authenticated user without permission gets 403, not 401.
- **Token/session:** we use stateless double-click tokens in the Authorization header — mobile-friendly, horizontally scalable, revocable. JWT is a config-change future upgrade but harder to revoke.
- **Password hashing:** salted, slow, one-way (PBKDF2 default in Django). Verify = hash input + compare. Plaintext never touches the DB.
- **Permissions:** DRF classes (`AllowAny` , `IsAuthenticated` , custom role classes) — server-side gate per endpoint.
- **Object-level authz:** queryset filter + `has_object_permission` ; per-row owner checks.
- **User ownership:** `save(owner=request.user)` ; client `owner` fields read-only; IDOR spoofing structurally impossible.
- **Input validation:** serializer = the security gate (types, sizes, ranges, lengths, file caps).
- **CORS/CSRF:** CORS is a browser rule (mobile ignores it); token-in-header means no cookie → CSRF classic vector gone.
- **Env vars/secrets:** never in code/git; DEBUG off in prod; rotation = env change + redeploy.

File Handling

- **multipart/form-data**: one request, file + fields, raw bytes streamed — no base64 bloat.
- **Cloudflare R2**: object storage, zero egress, S3-compatible, CDN-backed.
- **S3-compatible**: standard bucket/key/object contract; provider-agnostic code.
- **boto3**: low-level SDK for presigned URLs and direct object ops.
- **django-storages**: framework glue so model `ImageField` transparently uses R2.
- **Presigned URLs**: short-lived, scoped, one-action signed links — direct upload, expiring download.

Deployment

- **Git/GitHub:** feature branches → PR → review → merge to `main`; `.gitignore` keeps secrets out.
- **Render:** push → build → migrate → Gunicorn → health check green. Managed Postgres + env injection.
- **Production API:** DEBUG off, ALLOWED_HOSTS locked, secrets from env, Gunicorn (not runserver), HTTPS forced, throttled, health checks, error logging.
- **HTTPS:** TLS terminated at Render's edge (auto Let's Encrypt), SSL redirect + HSTS.
- **Environment variables:** the only place production secrets live; per-environment separation (dev vs prod).
- **Health checks:** unauthenticated `/health/` that actually pings the DB → 200 only when wired; Render auto-restarts on failure.

```
# health endpoint – checks a real dependency, not a warm-and-fuzzy 200
def health(request):
    try:
        connection.ensure_connection()
        return JsonResponse({"status": "ok"}, status=200)
    except OperationalError:
        return JsonResponse({"status": "unhealthy"}, status=503)
```

6 · Critical Demo Flow (memorize this script)

1. **User logs in** — Flutter → `POST /api/auth/login/` → Django checks the hash → returns a token.
2. **Backend authenticates** — every request carries the token; DRF resolves it to a user.
3. **User reports an issue** — Flutter builds multipart: image + lat/lng + description + category (e.g. pothole, broken streetlight, water leak).
4. **Django validates** — types, coordinate ranges, 5 MB cap. Bad input → 400 immediately.
5. **Image → object storage** — streamed to R2; only the URL lands in the DB.
6. **Metadata → PostgreSQL** — report row with `owner` (server-set), PostGIS `POINT`, `image_url`, status.
7. **AI analysis** — Celery worker scores the image: is it really the claimed issue? how severe?
8. **Duplicate detection** — PostGIS `ST_DWithin` ~30 m on recent reports → merge/link instead of spamming the authority.
9. **Severity / priority** — image analysis + location + category (main road + deep + pothole = P0).
10. **Department routing** — `ST_Contains` against district polygons + category → the right department.
11. **Authority receives issue** — ranked, deduplicated, verified issue in their dashboard + notification.

The 30-second spoken version

⚡ **Recite:** "The user logs in — Django hashes and checks the password, returns a token. Every call carries that token; DRF resolves the user and permission classes gate the endpoint. The user reports an issue: one multipart upload — image, lat/lng, description — over HTTPS. The serializer validates everything; garbage gets a 400 before touching a model. Ownership is injected server-side — always the authenticated user. The image streams to R2; Postgres stores only the URL. Off the request path, a worker runs AI (does the image match the claimed issue? severity), duplicate detection via `ST_DWithin`, and routing via `ST_Contains` against district polygons + category. The authority sees a ranked, deduped, verified issue — and the citizen gets notified when it's resolved, which brings them back."

7 · Mentor Probe Prep (the questions they ask beyond your list)

7.1 "Show me your code" — six spots to have ready

1. **models.py** — `ForeignKey(User)` (ownership), `PointField` (PostGIS), `ImageField` (R2), severity choices.
2. **serializers.py** — field types, max lengths, 5 MB cap, `read_only_fields = ("owner", "id", "severity")`.
3. **views.py viewset** — `get_queryset` filtered by `request.user`, `permission_classes`, `save(owner=request.user)`.
4. **The PostGIS query** — the `ST_DWithin` nearby/duplicate lookup.
5. **settings.py** — `DATABASES` (django.contrib.postgres), R2 storage env vars, `DEBUG` from env.
6. **/health/ endpoint** — the DB-connectivity check.

Pro move: know which *files* you can open in under 5 seconds. Fumbling for the viewset while a mentor waits is the #1 credibility killer.

7.2 Tradeoff questions (answer cold)

- **Django vs FastAPI/Flask** — "Batteries included — auth, ORM, migrations, security defaults — so we don't rebuild foundations. FastAPI wins on raw throughput/async; Django won on end-to-end build speed for a CRUD + AI app."
- **Postgres vs DynamoDB** — "DynamoDB scales horizontally beautifully, but we need relationships, transactions, and geo — none are its home turf. Postgres + PostGIS nails all three in one database."
- **R2 vs Firebase Storage** — "Firebase is quick to start but locks us into Google's stack and pricing; R2 is S3-compatible (portable code) with zero egress fees."
- **Token vs JWT** — "Opaque tokens: simplest, instantly revocable server-side. JWT's statelessness is attractive at massive scale — a config change to adopt, but harder to revoke."
- **"What would you cut if you rebuilt it?"** — "The admin panel and template layer — API-only product, clients never touch them. Keep everything else; security defaults are worth the weight."

7.3 Edge cases (pick two, know them brutally)

- **Duplicate reports** — "On create, `ST_DWithin` against recent unprocessed reports within ~30 m. If found we link/merge — increment an occurrence count — instead of spamming the authority. We link, we don't auto-delete; humans decide eventually."
- **Image that doesn't match the claimed issue** — "AI confidence below threshold → human-review queue. Misabeled or junk images never reach the authority's action feed. All uploads still pass validation first."
- **Oversized upload (DoS)** — "Serializer caps at 5 MB, multipart streams bytes, and DRF throttling limits requests per user per minute. Rejected with an early status code before buffering."
- **GPS spoofing** — "We validate ranges and bound trust: flag outliers, show reverse-geocoded location so the authority can sanity-check. We don't pretend to solve AR spoofing."
- **(0,0) / NaN coords** — "Serializer rejects NaN and coerces invalid flags; (0,0) — the Gulf of Guinea — routes to a manual triage queue instead of crashing routing."

7.4 Business one-liners (mentors often end here)

- **Customer:** "Two-sided — citizens report; municipal road authorities consume. The authority dashboard is where revenue lives."
- **Revenue:** "SaaS subscription per authority, tiered by population coverage + analytics. Citizens' side is free — they're the data source."

- **Why authorities pay:** "Auto-deduped, severity-ranked, GPS-precise, AI-verified issues — hours saved per week in their response teams."
- **Moat:** "Data density — more verified, deduplicated reports per region than anyone, plus routing models tuned to each jurisdiction."
- **Retention:** "The loop closes — 'your pothole was fixed' drives the next report. Authority wins, citizen wins, we get better data each cycle."

7.5 Sound evidence-based with numbers

- **Nearby at 1M rows:** "GiST spatial index → index-depth bound, milliseconds; a table-wide Haversine degrades linearly — the case for PostGIS in one sentence."
- **Image path:** "5 MB cap, streamed to R2; the DB stores a ~60-byte URL. Egress is free."
- **Upload UX:** "The API responds once metadata + storage commit; AI runs async on a worker — the user gets instant confirmation and we grade in the background."
- **Scaling:** "Stateless Django + token auth → add instances behind a load balancer. Checkpoints: DB (PgBouncer + replicas), workers (queue), image egress (free on R2)."

8 · Rapid-Fire One-Liners (final cheat sheet)

- **Why token auth, not session?** — Mobile apps don't do cookies well; tokens survive restarts, scale across servers, and are standard for API apps.
- **Why R2, not S3?** — S3-compatible (django-storages/boto3 unchanged) but no egress fees — a real cost win for image-heavy apps.
- **Why not Firebase?** — Great for prototyping; locks you in. Portable Django + Postgres + object storage is judge-impressive on architecture questions.
- **Health checks** — Render pings `/health/`, which checks the DB and returns 200; failure → auto-restart.
- **AuthN vs AuthZ** — AuthN answers *who are you?*; AuthZ answers *are you allowed?*
- **The trust rule** — ownership/roles are always derived server-side from the token; anything from the client body is treated as untrusted input.
- **The security-layers answer** — HTTPS (in transit) + hashing (at rest) + token auth (identity) + permissions/querysets (access) + serializer validation (input) + env secrets (configuration).
- **The golden thread** — Flutter ships a token over HTTPS; every call is authenticated and object-authorized; images stream to R2; metadata is written transactionally to Postgres with PostGIS thinking; Render + env vars + health checks keep it public, secret-safe, and self-healing.

End of master guide. Read twice, then drill each ⚡ **Say This** box out loud until it rolls off the tongue.